

Guess Market - Exercise 1

Bonus implemented

Saving and loading the system state (Java serialization). Menu options 6 and 7. The user gives a full path including the file name **without an extension**; `.gm` is appended by the engine. (fits mac and windows OS, note that windows path is with `"\"` so when asked to write full path to saved session it need to look like `"*\\PATH\\your_desired_file_name"`)

Submitter details

Name	Stav Hecht
ID number	211794151
Email	stavhe@mta.ac.il

Project repository

<https://github.com/stavhecht/GuessMarket>

Build and run

```
▶ ./run.sh      # compile Engine/src + UI/src into out/classes, then run ui.Main (macOS/Linux)
  run.bat      # the same on Windows
```

Plain `javac` over the two source roots with the JAXB jars from `lib/` on the classpath - no Maven/Gradle. Requires **JDK 17+** (records, switch expressions, `@Serial`). The scripts deliberately do not `cd`, so a relative path typed into the application resolves against the directory you launched it from.

Main classes

Class	Role
MarketEngine	Façade - the only engine class the UI imports, and the only place DTOs are built. One method per user command.

Class	Role
EventManager	Owns the loaded events in a <code>LinkedHashMap</code> (file order, keyed by id). Holds the entirety of the mutable state.
Event	Aggregate root - owns its two <code>Option</code> s, its market-maker <code>Account</code> and its trade history. All state changes go through it.
Option / Account / Trade	Outstanding shares per outcome; the market maker's balance and commission; one immutable purchase record.
LmsrCalculator	The market-maker maths: <code>prices</code> , <code>cost</code> ($C(q)$), <code>purchaseCost</code> , <code>initialSubsidy</code> . Pure and stateless.
XmlEventLoader	File $\rightarrow$ domain: open the <code>.xml</code> , unmarshal via JAXB, apply the rules, build <code>Event</code> s.
engine.schema	JAXB classes describing the file format only. No domain logic.
ConsoleApp / InputReader / OutputFormatter	Menu and dispatch; the only class that reads the console; the only class that writes to it.

Choices made

State lives in one object. `EventManager` holds everything mutable, so the bonus is a single `writeObject` / `readObject` rather than per-event serialization. Every model class pins `serialVersionUID = 1L` so save files stay loadable.

Failure is atomic by construction, not by try/catch. The loader throws before a single object reaches `EventManager`, so a rejected file leaves the previous session intact. `loadState` reassigns its field only after a successful read. `participate` validates and computes everything *before* the first mutation, so a rejected purchase can never issue shares against an untouched account.

One exception hierarchy, caught in one place. Everything a user can cause extends `EngineException` (unchecked). `ConsoleApp.dispatch` catches it exactly once and prints the message, so a rejected command returns to the menu instead of ending the session; the individual handlers contain no try/catch at all.

On-close commission comes out of the winners' payout, not the account. Each account is seeded with $b \cdot \ln 2$, the provable worst-case LMSR loss. Taking the operator's cut from money already owed to winners keeps that guarantee intact no matter how one-sided the market got - funding it from the balance could overdraw the account. `Account` also keeps `balance` and `commissionCollected` as separate pots, so the balance stays exactly equal to $C(q)$.